



IoTCore.Net Integration Guide

Version 6.0.0



CONTENTS

1	Scope	5
2	What is IoTCore?	5
2.1	The Object Model Layer	5
2.1.1	The Element	5
2.1.2	The Element Tree	5
2.2	The Communication Layer	6
2.3	Well-Known Services	6
2.4	Profiles	6
3	What is the IoTCore.Net?	6
4	License	6
5	Terms, Definitions, and Conventions	6
5.1	IoTCore	6
5.2	Element	6
5.3	Element Tree	6
5.4	Profile	7
5.5	Command API	7
5.6	Programming API	7
5.7	Network Adapter	7
5.8	Protocol	7
5.9	Variant	7
6	IoTCore Concepts	7
6.1	The IoTCore Architecture	7
6.2	Using the IoTCore	8
6.3	Developing the IoTCore	8
6.4	Developing an IoTCore Application	8
6.5	Modelling Data, Services and Events	9
6.5.1	Basic Element Properties	9
6.5.2	The Device Element	10
6.5.3	The Structure Element	10
6.5.4	The Service Element	10
6.5.5	The Data Element	10
6.5.6	The Event Element	11
6.5.7	The Element Format	11
6.6	Addressing Elements	14
6.7	The command API	14

6.7.1	Error Handling.....	16
6.8	Sending Events	17
7	Variant – Message Data Representation.....	18
7.1	Variant Class Hierarchy	19
7.2	Converting between Variant and object.....	19
7.2.1	The Variant Property Attribute.....	20
7.2.2	The Variant Contract Resolver.....	20
8	The IoTCore.Net Programming API	20
8.1	Create an IoTCore Application	20
8.2	Add Elements to the IoTCore Element Tree	20
8.2.1	Add a structure element.....	21
8.2.2	Add a service element	21
8.2.3	Add an event element	23
8.2.4	Add a data element	23
8.2.5	Add a simple data element.....	29
8.3	Handle Errors in Service and Data Element Handler Functions.....	29
8.4	Removing an Element	30
8.5	Getting Elements by Address.....	31
8.6	Add User Specific Data to Elements.....	31
8.7	Subscribe to Events.....	32
8.7.1	Add an event handler to the “treechanged” event of the IoTCore.....	32
8.7.2	Add an event handler to an event element.....	33
8.8	Raise Events	34
8.9	Create Links to Elements.....	35
8.10	Logging	35
8.11	Persistence.....	36
8.12	Create Network Adapters	36
8.12.1	Create a server network adapter	36
8.12.2	Create a client network adapter factory	36
8.12.3	Create a Http Server Network Adapter	36
8.13	Create and Load an IoTCore Profile	37
8.13.1	Customize and Load the “deviceinfo” Profile	37
8.13.2	Create and Load your own Profile.....	38
8.13.3	Create the Monitor Profile	41
8.14	Secure Communication.....	42
8.14.1	Https.....	42

8.14.2 Mqtts..... 42

1 SCOPE

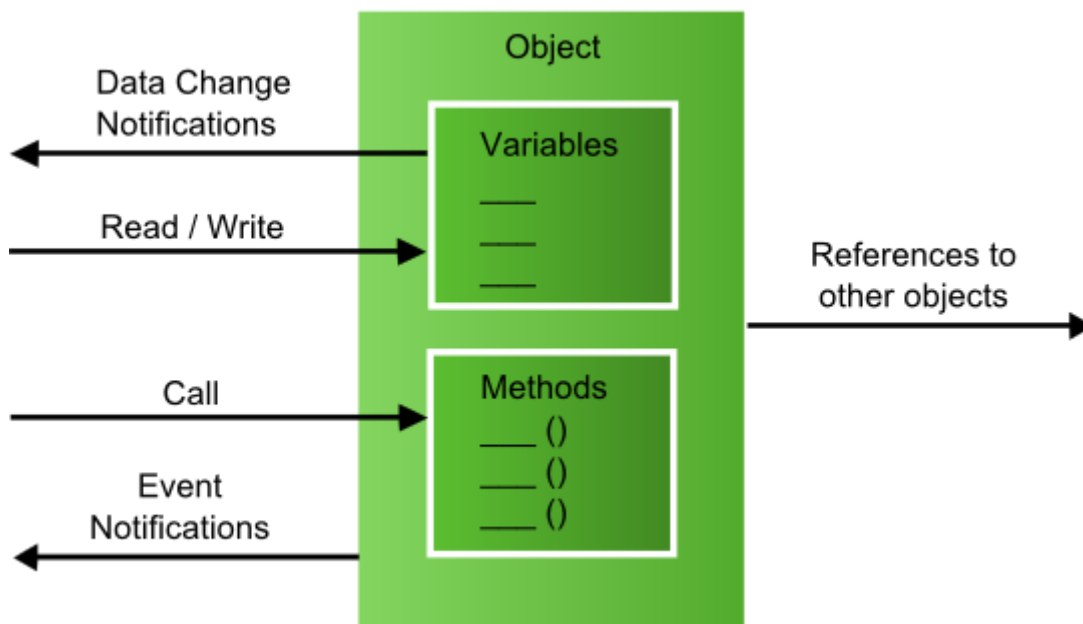
This document describes the IoTCore.Net programming API. This API is used by integrators to develop an IoTCore application which uses the IoTCore.Net libraries to represent their system according to the IoTCore specification. The document also gives a brief introduction into the concepts of the IoTCore specification and the architecture of the IoTCore.Net application framework.

2 WHAT IS IOTCORE?

The IoTCore is a specification that defines how a system represents its functionality as objects in a hierarchical structure and how these objects are made accessible to other systems. The specification comprises two layers that define the functional modelling of a system and the communication of that system with other systems.

2.1 THE OBJECT MODEL LAYER

Generally the functionality of an object falls into three categories, variables, methods, and notifications.



2.1.1 The Element

In IoTCore these objects are called "element". It defines elements in terms of "data elements" which represent variables, "service elements" which represent methods, and "event elements" which represent notifications. These elements can be nested to create a full featured object.

2.1.2 The Element Tree

In IoTCore the elements are organized in a hierarchical tree structure, which therefore is called "element tree". A "structure element" groups logically related elements together. References relate elements to each other. A "device element" is defined as the root element of the element tree which provides well-known services for browsing and querying the element tree as well as the IoTCore application itself.

2.2 THE COMMUNICATION LAYER

The IoTCore defines the communication layer in terms of network adapters and protocols. The network adapters can be seen as the transport layer whereas the protocol is the format in which the data is transferred.

2.3 WELL-KNOWN SERVICES

The IoTCore specification also defines some root level services that all IoTCore applications must provide. These are mainly services for browsing and querying the element tree, as well as services for querying or configuring the IoTCore application itself.

2.4 PROFILES

In addition to the IoTCore specification itself several companion specifications, called “profiles”, are being developed that define how well-known and frequently used scenarios and use cases shall be represented in an IoTCore application.

3 WHAT IS THE IOTCORE.NET?

The IoTCore.Net is an application framework for .Net that implements the IoTCore specification as a set of classes and services. It is delivered as set of libraries (as nuget packages) that implement the IoTCore specification, some libraries that include network adapters and protocols, and some libraries with implementations of frequently used profiles.

4 LICENSE

The IoTCore.Net implementation is an open source project available at [GitHub - ifm/iot-core: ifm IoTCore](https://github.com/ifm/iot-core).

5 TERMS, DEFINITIONS, AND CONVENTIONS

5.1 IOTCORE

The IoTCore is a specification that defines how a system represents its functionality as objects in a hierarchical structure and how these objects are made accessible to other systems.

In common speech the term IoTCore is used for the IoTCore specification, as well as for the IoTCore libraries that implement the specification and also for the application that integrates the IoTCore libraries.

5.2 ELEMENT

An element represents an object. It can be of type data, service, event, structure, and device. The device element is always the root element of the element tree and there can be only one.

5.3 ELEMENT TREE

The element tree is the hierarchical structure how elements are organized.

5.4 PROFILE

A profile is a contract that specifies how well-known and frequently used scenarios and use cases shall be represented in an IoTCore application.

5.5 COMMAND API

The command API, also called “Network API” is a message based interface which is typically used by network adapters to invoke services of the IoTCore elements.

5.6 PROGRAMMING API

The programming API, also called “Firmware API”, is the interface applications use to model their functionality and data and to load network adapters and profiles into their application.

5.7 NETWORK ADAPTER

A network adapter is an implementation of a network interface that serves as adapter between the IoTCore and external systems.

5.8 PROTOCOL

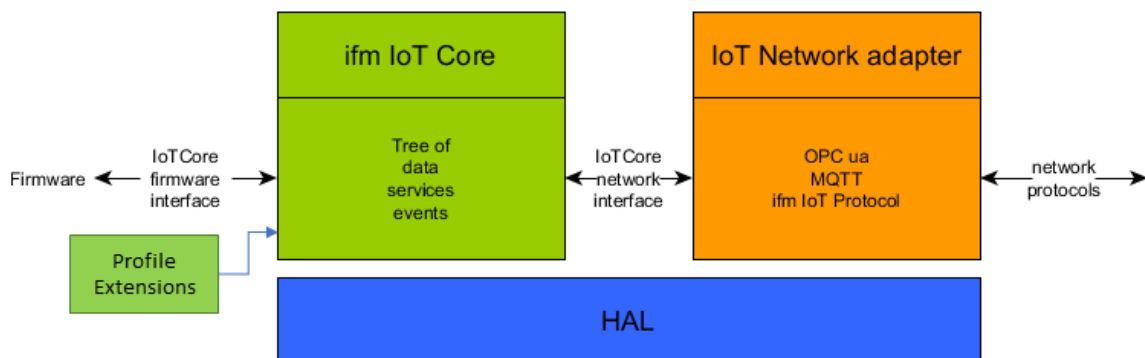
The protocol defines how data that is transferred between a network adapter and an external system is formatted.

5.9 VARIANT

A Variant is a general purpose container class used for transporting data from the command API to a service element handler function and back.

6 IOTCORE CONCEPTS

6.1 THE IOTCORE ARCHITECTURE



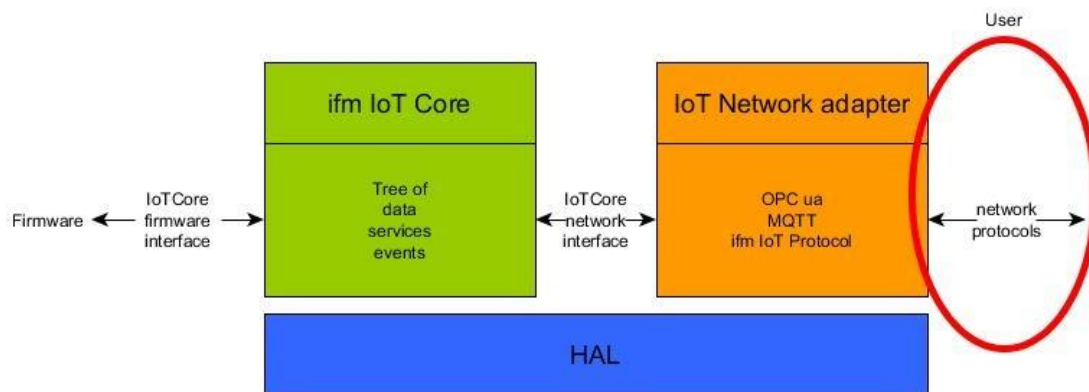
The IoTCore itself implements just a basic set of standard services and elements, but it provides several interfaces through which an application can extend the functionality of the IoTCore.

The standard set of services and elements provided by the IoTCore can be extended by the application by adding its own data, service, and event elements and it can also load already implemented and ready to use assemblies that implement well known and widely used profiles.

The IoTCore network adapter interfaces provide interfaces to load network adapter clients and servers. Each network adapter implements its own communication interface and protocol. The network adapter servers provide the application's services to external systems, whereas the network adapter clients enable the IoTCore to communicate with other external systems, like other IoTCores or MQTT message brokers.

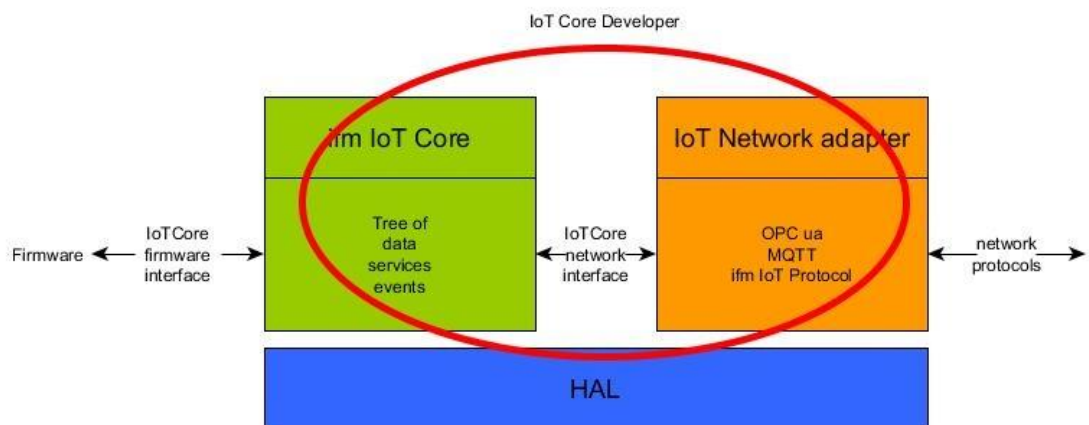
6.2 USING THE IoTCORE

Users of an IoTCore application communicate with the IoTCore via the interfaces and protocols that the network adapters provide. Users can be, for instance, other applications that control the IoTCore application or websites that allow monitoring the status of the IoTCore application.



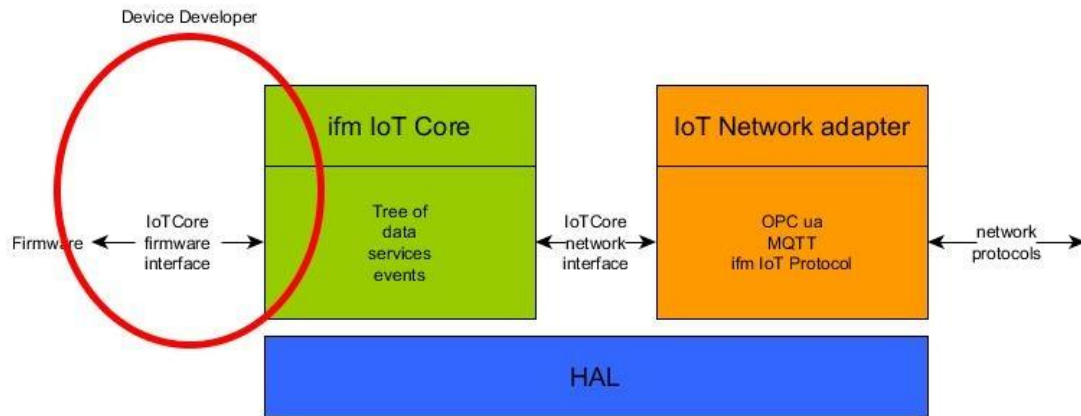
6.3 DEVELOPING THE IoTCORE

The IoTCore developers naturally work on the IoTCore, network adapters, protocols, and profiles. As the IoTCore supports an open architecture, every user can write its own set of profiles or network adapters for various communication hardware, transport layers, and protocols.



6.4 DEVELOPING AN IoTCORE APPLICATION

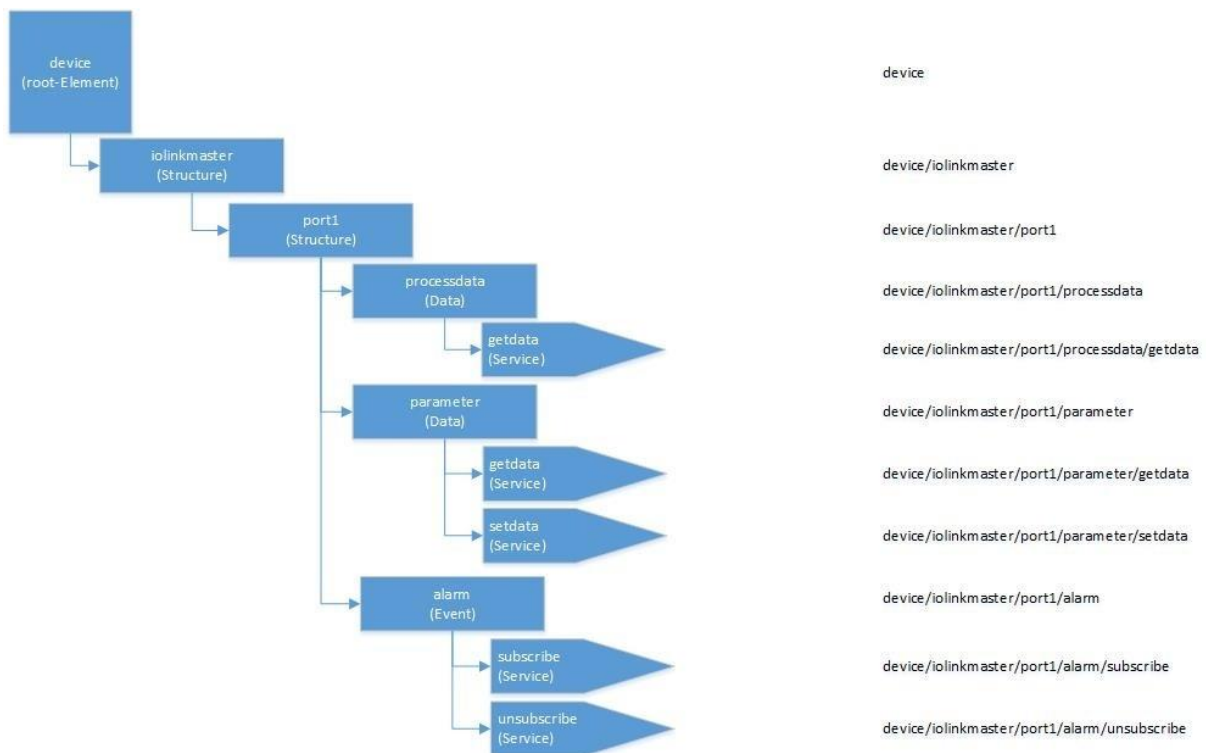
Developers of an IoTCore application typically implement the data, service, and events for an application and create the corresponding elements in the element tree, to make the elements accessible via the command API.



6.5 MODELLING DATA, SERVICES AND EVENTS

The purpose of the IoTCore is to represent the data and functionality of an application and make it available to other applications. The application is modeled as a structured tree of elements, where each element can be of type service, data, event, or structure with a device element at the root of the element tree. The name “device” has historical reasons, actually an IoTCore application can be anything and is not limited to a kind of physical device.

This view of an application shows a tree of elements describing the functionality of the application in form of data, services, and events, using structure elements to organize the tree.



6.5.1 Basic Element Properties

All elements share the same basic properties.

Property	Mandatory	Data type	Description
----------	-----------	-----------	-------------

type	yes	String	The type of the element.
identifier	yes	String	The identifier of the element.
format	no	Format	The format specification of the element.
profiles	no	String[]	The profiles the element belongs to.
uid	no	String	The unique identifier of the element.
subs	no	Element[]	The child elements of the element.

6.5.2 The Device Element

Each IoTCore has one element of type “device” which serves as the root element of the element tree. The device element has the following defined child elements:

Element	Mandatory	Element type	Description
getidentity	yes	Service	Gets identity information of the application.
gettree	yes	Service	Gets the element tree.
deviceinfo	yes	Structure	The deviceinfo profile provides elements that describe the device.
querytree	no	Service	Queries the element tree for elements that match a specific filter criteria.
getdatamulti	no	Service	Gets the value of multiple data elements.
setdatamulti	no	Service	Sets the value of multiple data elements.
getsubscriberlist	no	Service	Gets the subscribers from a particular event element.
treechanged	no	Event	Raises a tree-changed event.

6.5.3 The Structure Element

The element of type “structure” serves as structural element that organizes logically related elements into groups. It has no defined child elements.

6.5.4 The Service Element

The element of type “service” serves as the executing element for service requests. It has no defined child elements.

6.5.5 The Data Element

The element of type “data” represents a variable. The data element has the following defined child elements:

Element	Mandatory	Element type	Description
---------	-----------	--------------	-------------

getdata	no	Service	Gets the value from the element.
setdata	no	Service	Sets the value to the element.
datachanged	no	Event	Raises a data-changed event.

6.5.6 The Event Element

The element of type “event” can raise notifications asynchronously. It supports adding and removing subscriptions to the event. The event element has the following defined child elements:

Element	Mandatory	Element type	Description
subscribe	yes	Service	Adds a subscription to the element.
unsubscribe	yes	Service	Removes a subscription from the element.
triggerevent	no	Service	Allows to raise the event from the command API.

6.5.7 The Element Format

6.5.7.1 The Data Element Format

The format property of a data element describes the type and format of the element value.

Format type:

Property	Mandatory	Data type	Description
type	yes	String	The type of the data.
namespace	no	String	The namespace context of the format.
encoding	no	String	The data encoding.
valuation	no	Valuation	The data valuation.

Types of formats:

Type	Encoding	Valuation
number	Integer (signed 32 bit)	IntegerValuation
	Int8 (signed 8 bit)	Int8Valuation
	UInt8 (unsigned 8 bit)	UInt8Valuation
	Int16 (signed 16 bit)	Int16Valuation
	UInt16 (unsigned 16 bit)	UInt16Valuation
	Int32 (signed 32 bit)	Int32Valuation

	UInt32 (unsigned 32 bit)	UInt32Valuation
	Int64 (signed 64 bit)	Int64Valuation
	UInt64 (unsigned 8 bit)	UInt64Valuation
	Float (single precision, 32 bit)	FloatValuation
	Double (double precision, 64 bit)	DoubleValuation
string	utf-8, hexstring, base64	StringValuation
boolean		
enum	Integer	IntegerEnumValuation
binary	base64	
array		ArrayValuation
object		ObjectValuation
map		MapValuation
self		
any		

[U]Int[Type]Valuation type

Property	Mandatory	Data Type	Meaning
min	no	As specified in format	The minimum value.
max	no	As specified in format	The maximum value.

FloatValuation, DoubleValuation type

Property	Mandatory	Data Type	Meaning
min	no	As specified in format	The minimum value.
max	no	As specified in format	The maximum value.
decimals	no	int	The number of decimals.

StringValuation type

Property	Mandatory	Data Type	Meaning
minlength	no	Integer	The minimum string length.
maxlength	no	Integer	The maximum string length.
pattern	no	String	The regular expression to evaluate string.

IntegerEnumValuation type

Property	Mandatory	Data Type	Meaning
valuelist	yes	<String:String>[]	The value:text pairs that define possible values. Note that the value has to be a number in string format!

ArrayValuation type

Property	Mandatory	Data Type	Meaning
length	no	Integer	The number of items in the array.
format	yes	Format	The format of the array item.

ObjectValuation type

Property	Mandatory	Data Type	Meaning
fields	yes	Field[]	The field items of the object.

Field type

Property	Mandatory	Data Type	Meaning
name	yes	String	The name of the field.
optional	no	Boolean	true if field is optional; false if the field is mandatory.
format	no	Format	The format of the field.

MapValuation type

Property	Mandatory	Data Type	Meaning
----------	-----------	-----------	---------

format	yes	Format	The format of the values in the map. The keys are always of type string.
--------	-----	--------	--

6.5.7.2 The Service Element Format

The format property of a service element describes the type and format of the service input and output data.

Format type:

Property	Mandatory	Data type	Description
input	no	InputFormat	The type of the service input data.
output	no	Format	The type of the service output data.

InputFormat type:

Property	Mandatory	Data type	Description
format	yes	Format	The type of the service input data.
optional	no	Boolean	If true, providing the input data is optional, i.e. the service can be called without data; otherwise providing the data is mandatory. The default is false.

6.6 ADDRESSING ELEMENTS

Each element in the IoTCore is referenced by its address. The address of an element is constructed from its position in the element tree. The address of an element is the address of its parent joined with its own identifier, separated by a '/' (slash). To guarantee unique addresses for the elements the element identifier must be unique among the children of an element. Using the name of the root device element in the address is optional.

Examples:

device/iolinkmaster/port1/processdata/getdata

/iolinkmaster/port1/processdata/getdata

6.7 THE COMMAND API

The IoTCore provides a message based interface, that network adapters use to invoke services from the elements in the element tree. The command interface supports a synchronous request-response communication model. Only service elements can process a request, therefore only service elements can be addressed. When the IoTCore receives a request message it looks up the addressed service element and passes the payload from the request message to that service element for processing. The result of the service call is returned in the payload of the response message.

Request message type

Field	Mandatory	Data Type	Meaning
code	Yes	Integer/String	Request type (string is deprecated)
cid	Yes	Integer	A unique context ID, provided in the request and returned in the corresponding response
adr	Yes	String	Address of the requested service element
data	No	Object	The payload of the service request
reply	No	String	The address for the response, if provided
auth	No	AuthenticationInfo	Authentication information for the service request

The message code for a request message is defined as

Code as string	Code as number	Meaning
request	10	The message is a request message

AuthenticationInfo type

Parameter	Mandatory	Data Type	Meaning
user	yes	String	The username (base64 encoded)
passwd	no	String	The password (base64 encoded)

Response message type

Field	Mandatory	Data Type	Meaning
code	yes	Integer	The result code of the executed service
cid	yes	Integer	A unique context ID, provided in the request and returned in the corresponding response
adr	yes	String	The address of the requested service element or the reply address
data	no	Object	The payload of the service response

Response codes

Code	Msg (examples)	Description
200	Success	The service executed successfully
230	Success, reboot required	The service executed successfully and a reboot is required
400	Bad request	The service request is invalid or malformed
401	Access denied	Authentication is required and has failed or has not been provided
404	Element not found	The requested service does not exist
416	Data out of range	The provided service data is out of range
422	Data invalid	The provided service data payload is invalid
423	Locked	The service is locked, disabled or busy
424	Failed dependency	A dependency is missing to execute the service
500	Internal error	An error condition, that is not considered and handled by IoTCore and therefore raised by the underlying runtime
501	Not implemented	The service is not implemented
503	Not available	The service is currently not available
504	Timeout	Access to a resource timed out
550	Service failed	The service execution failed
901	Already exists	The element already exists

6.7.1 Error Handling

If the service call executed successfully the code field in the response message is set to 200 (Success) and the response message data field holds the service result.

If the IoTCore could not invoke the service, for instance, because the addressed element does not exist or is not a service or is currently locked, or the message data is malformed, the IoTCore returns one of the error codes defined above in the code field of the response message. The data field of the response message is empty.

For invalid user input the following rules apply:

- If something is wrong inside the message, so that the message can't be processed the response code is **400** (Bad Request). This is for instance if a mandatory field is missing, the message format is bad or there is a data type mismatch like code is a string, cid is a string or adr is an integer.

- If there is no element with the given adr the response code is **404** (Not Found). We do not verify if the adr is a valid address format, for instance adr is `"/hi///ho///a!"$$"/` the response code is also 404. This is, because the request was valid, but there is no such element. This helps differentiate between a badly formed message and a bad address and the caller is directly told to have a look at the address.
- If the provided data is outside of the allowed value range the response code is **416** (Data Out of Range). For instance the allowed range for a parameter is 0-100, but the provided value is 200.
- If anything within the message data field is bad the response code is **422** (Data Invalid). For example, a mandatory field is missing, a field has the wrong data type or the provided data is invalid for the service.

If the service could be invoked by the IoTCore, but the service execution failed, the service can set the code field in the response message one of the error codes defined above or, if non applies, to the generic code 550 (Service failed). Additional service specific error information can be returned in the data field of the response message. The service specific error information is defined as

The ErrorInfo type

Parameter	Mandatory	Datatype	Description
code	no	Number	Service specific error code
msg	no	String	Service specific error message (What happened)
details	no	String	Additional and more detailed information (Why did it happen and how to fix the problem)

6.8 SENDING EVENTS

When a client has subscribed to an event and the event is raised, the IoTCore can send an event message to the recipient of the event as specified in the subscription to the event.

Event message type

Field	Mandatory	Data Type	Meaning
code	yes	Integer/String	Event type (string is deprecated)
cid	yes	Integer	The subscription id, if provided in the subscription or the cid used in the subscription
adr	yes	String	The target address for the event as specified in the subscribe request
data	no	EventInfo	The event data

The message code for an event message is defined as

Code as string	Code as number	Meaning
event	80	The message is an event message

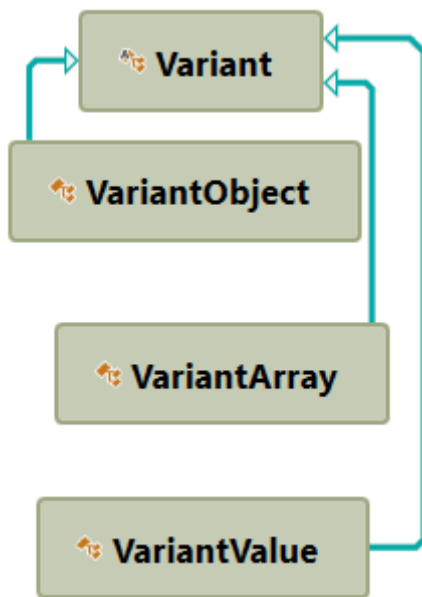
EventInfo type

Parameter	Mandatory	Data Type	Meaning
srcurl	yes	String	The address of the event element, that sends the event
eventno	yes	Integer	Number of the event
payload	no	<String:Variant>[]	Data element address:value pairs, as requested in the subscription
subscribeid	yes	Integer	The identifier for the subscription

7 VARIANT – MESSAGE DATA REPRESENTATION

The IoTCore provides an open architecture that allows applications to implement their own service and data accessor functions. These functions can define their own service specific data types for incoming and outgoing data which are, of course, unknown at compile time of the IoTCore application framework. Therefore the IoTCore application framework cannot provide type-safe classes for these data types. As these functions need to exchange data via messages with the command API, the message payload is defined as a Variant type. A Variant and its derived types are general purpose data container classes used to transfer the request message payload to service handler function and the response message payload back to the caller.

7.1 VARIANT CLASS HIERARCHY



Variant type

The class “Variant” is an abstract base class which is at the root of the inheritance chain.

VariantValue type

The class “VariantValue” can hold values for simple data types like int, float, string, byte, char and others.

VariantArray type

The class “VariantArray” can hold arrays of simple types and “VariantValue”.

VariantObject type

The class “VariantObject” can hold complex types. A “VariantObject” manages the values as a dictionary of “Variant” types. It supports deep nesting of all types of “Variant”.

7.2 CONVERTING BETWEEN VARIANT AND OBJECT

The Variant class provides two static methods that allow converting objects to a Variant and back

```
public static Variant FromObject(object data)
```

```
public static T ToObject<T>(Variant data)
```

There are restrictions on the object type **T** that should be converted to and from a Variant:

- The type must have a public parameterless constructor.
- Currently only array, list and dictionary collection classes are supported.
- Recursive objects are not supported.

ToDo: Provide example

The Variant class library provides attribute classes that allow some control over how the conversion between object and Variant is done.

7.2.1 The Variant Property Attribute

ToDo: Explain. Provide example

7.2.2 The Variant Contract Resolver

ToDo: Explain. Provide example

8 THE IOTCORE.NET PROGRAMMING API

8.1 CREATE AN IOTCORE APPLICATION

To get access to the IoTCore.Net libraries you need to add the “ifm.IoTCore.Sdk.[release]” nuget package to your application project. This package includes the IoTCore.Net base libraries as well as the HTTP and MQTT network adapters and the Device-Info and Monitor profiles. If you need additional network adapters or profiles also add the according nuget packages.

The first thing you need to do is to create a new instance of the IoTCore class by calling the “IoTCoreFactory.Create()” method which returns an “IIoTCore” instance. The method takes a name as parameter and passes it to the constructor of the IoTCore class. The constructor of the IoTCore class internally creates a device element member, which serves as the root element for the IoTCore element tree. The constructor of the IoTCore class passes this name to the root device element as its identifier. You can access the properties of the IoTCore via the instance returned by the create method.

```
namespace Sample01;

using System;
using ifm.DataStore;
using ifm.Logger;
using ifm.IoTCore;

internal class Program
{
    private static void Main()
    {
        try
        {
            var iotCore = IoTCoreFactory.Create("MyIoTCore",
                new DataStore("MyIoTCore.txt"),
                new NullLogger());

            Console.WriteLine(iotCore.ApiVersion);
            Console.WriteLine(iotCore.Identifier);
        }
        catch (Exception e)
        {
            Console.WriteLine(e.Message);
        }
        Console.ReadLine();
    }
}
```

8.2 ADD ELEMENTS TO THE IOTCORE ELEMENT TREE

To add an element to the element tree, use the “ElementManager” property from the IoTCore instance and call one of the “CreateElement” methods. The “CreateElement” methods return an instance of the corresponding element class, like “IStructureElement”, “IServiceElement”, “IDataElement”, “IEventElement”. The “CreateElement” methods take as first parameter the parent element, to which you want to add the newly created element. It has additional parameters for the various element properties like format or profiles. It also has a parameter that defines whether the

“treechanged” event of the parent element is raised or not. If you want to make multiple changes to the element tree, it is much more efficient to add or remove elements without raising the tree changed event for each change, because this would trigger all registered event handler each time. Instead, consider making all changes without raising the event and when all is done raise the event once by calling the element manager “RaiseTreeChanged” method.

8.2.1 Add a structure element

A structure element serves as a container in which elements that semantically belong together are organized.

```
namespace Sample02;

using System;
using System.Collections.Generic;
using ifm.DataStore;
using ifm.Logger;
using ifm.IoTCore;

internal class Program
{
    private static void Main()
    {
        try
        {
            var ioTCore = IoTCoreFactory.Create("MyIoTCore",
                new DataStore("MyIoTCore.txt"),
                new NullLogger());

            ioTCore.ElementManager.CreateStructureElement(ioTCore.Root,
                "struct1",
                null,
                ["profile1"]);
        }
        catch (Exception e)
        {
            Console.WriteLine(e.Message);
        }
        Console.ReadLine();
    }
}
```

8.2.2 Add a service element

The service element wraps the service handler function which executes the service. The service handler function can receive incoming and return outgoing data. The data type of the object that is passed into the service handler function or returned from it is of specified type in the template parameters. The service handler receives a reference to the service element from which it is called and the optional service information. For different service handler input and output parameters there are different service element types available. The “IActionServiceElement” type defines a service handler function that receives no input data and returns no output data. The “IGetterServiceElement” type defines a service handler function that has no incoming data but returns outgoing data. The “ISetterServiceElement” type defines a service handler function that has incoming data but returns no outgoing data. The “IServiceElement” type defines a service handler function that has incoming data and returns outgoing data.

```
namespace Sample03;

using System;
using ifm.DataStore;
using ifm.Logger;
using ifm.IoTCore;
using ifm.IoTCore.ElementManager.Contracts.Elements;

internal class Program
```

```

{
    private static void Main()
    {
        try
        {
            var iotCore = IoTCoreFactory.Create("MyIoTCore",
                new DataStore("MyIoTCore.txt"),
                new NullLogger());

            var struct1 = iotCore.ElementManager.CreateStructureElement(iotCore.Root,
                "struct1");

            iotCore.ElementManager.CreateActionServiceElement(struct1,
                "service1",
                HandleService1);
            iotCore.ElementManager.CreateGetterServiceElement<int>(struct1,
                "service2",
                HandleService2);
            iotCore.ElementManager.CreateSetterServiceElement<string>(struct1,
                "service3",
                HandleService3);
            iotCore.ElementManager.CreateServiceElement<string, string>(struct1,
                "service4",
                HandleService4);
            iotCore.ElementManager.CreateServiceElement<UserData, UserData>(struct1,
                "service5",
                HandleService5);
        }
        catch (Exception e)
        {
            Console.WriteLine(e.Message);
        }
        Console.ReadLine();
    }

    private static void HandleService1(IBaseElement element)
    {
        Console.WriteLine("Do something");
    }

    private static int HandleService2(IBaseElement element)
    {
        Console.WriteLine("Do something and return result");
        return 0;
    }

    private static void HandleService3(IBaseElement element, string data)
    {
        Console.WriteLine($"Do something with {data}");
    }

    private static string HandleService4(IBaseElement element, string data)
    {
        Console.WriteLine($"Do something with {data} and return result");

        return $"Received {data}";
    }

    private static UserData HandleService5(IBaseElement element, UserData data)
    {
        Console.WriteLine($"Do something with {data} and return result");
        return data;
    }
}

internal class UserData : IEquatable<UserData>
{
    public int Int1 = 10;
    public float Float1 = 1.2345f;
    public string String1 = "Hallo";

    public bool Equals(UserData other)
    {

```

```

        if (other == null) return false;
        return Int1 == other.Int1 &&
            Float1.Equals(other.Float1) &&
            String1 == other.String1;
    }

    public override string ToString()
    {
        return $"Int1={Int1} Float1={Float1} String1={String1}";
    }

    public override bool Equals(object obj)
    {
        return Equals(obj as UserData);
    }

    public override int GetHashCode()
    {
        return GetHashCode.Combine(Int1, Float1, String1);
    }
}

```

8.2.3 Add an event element

You can subscribe to an event element to receive notifications when the event is raised.

```

namespace Sample04;

using System;
using ifm.DataStore;
using ifm.Logger;
using ifm.IoTCore;

internal class Program
{
    private static void Main()
    {
        try
        {
            var ioTCore = IoTCoreFactory.Create("MyIoTCore",
                new DataStore("MyIoTCore.txt"),
                new NullLogger());

            var struct1 = ioTCore.ElementManager.CreateStructureElement(ioTCore.Root,
                "struct1");

            var event1 = ioTCore.ElementManager.CreateEventElement(struct1,
                "event1");
            event1.EventRaised += (sender, args) =>
            {
                Console.WriteLine("event1 raised");
            };

            event1.Raise();
        }
        catch (Exception e)
        {
            Console.WriteLine(e.Message);
        }
        Console.ReadLine();
    }
}

```

8.2.4 Add a data element

The data element class constructor takes the service handler functions for the getter and setter methods in its constructor, as well as a flag that specifies whether a “datachanged” event element is created or not. The service handler functions are provided by the application. A data element exists as read-only, write-only and read-write object.

The data element maintains an internal data cache. A cache timeout value can be provided in the data element constructor. If the cache timeout is not expired the data element getter function returns the value from the cache. If the timeout is expired the getter function is called, the cache is updated and the value is returned. If the setter function updates the value of the data element and the value has changed from the value in the cache a “datachanged” event is raised.

The data type of the object that the data element represents is specified as a template parameter of the data element class. If no format object is passed into the constructor, the object will create a format object, which specifies the type of the value. For more complex format specifications, including valuations, a format object must be created and passed into the constructor.

8.2.4.1 Add a bool data element

```
namespace Sample05;

using System;
using ifm.DataStore;
using ifm.Logger;
using ifm.IoTCore;
using ifm.IoTCore.ElementManager.Contracts.Elements;
using ifm.IoTCore.ElementManager.Contracts.Elements.Formats;

internal class Program
{
    private static void Main()
    {
        try
        {
            var ioTCore = IoTCoreFactory.Create("MyIoTCore",
                new DataStore("MyIoTCore.txt"),
                new NullLogger());

            var struct1 = ioTCore.ElementManager.CreateStructureElement(ioTCore.Root,
                "struct1");

            ioTCore.ElementManager.CreateDataElement<bool>(struct1,
                "bool1",
                GetBool1,
                SetBool1,
                format: new BooleanFormat());
        }
        catch (Exception e)
        {
            Console.WriteLine(e.Message);
        }
        Console.ReadLine();
    }

    private static bool GetBool1(IBaseElement element)
    {
        return _bool1;
    }

    private static void SetBool1(IBaseElement element, bool value)
    {
        _bool1 = value;
    }

    private static bool _bool1 = true;
}
```

8.2.4.2 Add an integer data element

```
namespace Sample06;

using System;
using ifm.DataStore;
using ifm.Logger;
using ifm.IoTCore;
using ifm.IoTCore.ElementManager.Contracts.Elements;
```



```

using ifm.IoTCore.ElementManager.Contracts.Elements.Formats;
using ifm.IoTCore.ElementManager.Contracts.Elements.Valuations;

internal class Program
{
    private static void Main()
    {
        try
        {
            var iotCore = IoTCoreFactory.Create("MyIoTCore",
                new DataStore("MyIoTCore.txt"),
                new NullLogger());

            var struct1 = iotCore.ElementManager.CreateStructureElement(iotCore.Root,
                "struct1");

            iotCore.ElementManager.CreateDataElement<int>(struct1,
                "int1",
                GetInt1,
                SetInt1,
                format: new Int32Format(new Int32Valuation(0, 100)));
        }
        catch (Exception e)
        {
            Console.WriteLine(e.Message);
        }
        Console.ReadLine();
    }

    private static int GetInt1(IBaseElement element)
    {
        return _int1;
    }

    private static void SetInt1(IBaseElement element, int value)
    {
        _int1 = value;
    }

    private static int _int1 = 10;
}

```

8.2.4.3 Add a float data element

```

namespace Sample07;

using System;
using ifm.Common;
using ifm.DataStore;
using ifm.Logger;
using ifm.IoTCore;
using ifm.IoTCore.ElementManager.Contracts.Elements;
using ifm.IoTCore.ElementManager.Contracts.Elements.Formats;
using ifm.IoTCore.ElementManager.Contracts.Elements.Valuations;

internal class Program
{
    private static void Main()
    {
        try
        {
            var iotCore = IoTCoreFactory.Create("MyIoTCore",
                new DataStore("MyIoTCore.txt"),
                new NullLogger());

            var struct1 = iotCore.ElementManager.CreateStructureElement(iotCore.Root,
                "struct1");

            iotCore.ElementManager.CreateDataElement<float>(struct1,
                "float1",
                GetFloat1,
                SetFloat1,
                format: new FloatFormat(new FloatValuation(-9.9f, 9.9f, 3)));
        }
    }
}

```

```

        catch (Exception e)
        {
            Console.WriteLine(e.Message);
        }
        Console.ReadLine();
    }

    private static float GetFloat1(IBaseElement element)
    {
        return _float1;
    }

    private static void SetFloat1(IBaseElement element, float value)
    {
        if (_float1.EqualsWithPrecision(value)) return;
        _float1 = value;
    }
    private static float _float1 = 1.2345f;
}

```

8.2.4.4 Add a string data element

```

namespace Sample08;

using System;
using ifm.DataStore;
using ifm.Logger;
using ifm.IoTCore;
using ifm.IoTCore.ElementManager.Contracts.Elements;
using ifm.IoTCore.ElementManager.Contracts.Elements.Formats;
using ifm.IoTCore.ElementManager.Contracts.Elements.Valuations;

internal class Program
{
    private static void Main()
    {
        try
        {
            var iotCore = IoTCoreFactory.Create("MyIoTCore",
                new DataStore("MyIoTCore.txt"),
                new NullLogger());

            var struct1 = iotCore.ElementManager.CreateStructureElement(iotCore.Root,
                "struct1");

            iotCore.ElementManager.CreateDataElement<string>(struct1, "string1",
                GetString1,
                SetString1,
                format: new StringFormat(new StringValuation(0, 100)));
        }
        catch (Exception e)
        {
            Console.WriteLine(e.Message);
        }
        Console.ReadLine();
    }

    private static string GetString1(IBaseElement element)
    {
        return _string1;
    }

    private static void SetString1(IBaseElement element, string value)
    {
        _string1 = value;
    }
    private static string _string1 = "Hallo";
}

```

8.2.4.5 Add an integer-enumeration data element

```

namespace Sample09;

```

```

using System;
using System.Collections.Generic;
using ifm.DataStore;
using ifm.Logger;
using ifm.IoTCore;
using ifm.IoTCore.ElementManager.Contracts.Elements;
using ifm.IoTCore.ElementManager.Contracts.Elements.Formats;
using ifm.IoTCore.ElementManager.Contracts.Elements.Valuations;

internal class Program
{
    private static void Main()
    {
        try
        {
            var ioTCore = IoTCoreFactory.Create("MyIoTCore",
                new DataStore("MyIoTCore.txt"),
                new NullLogger());

            var struct1 = ioTCore.ElementManager.CreateStructureElement(ioTCore.Root,
                "struct1");

            ioTCore.ElementManager.CreateDataElement<int>(struct1,
                "enum1",
                GetEnum1,
                SetEnum1,
                format: new IntegerEnumFormat(new IntegerEnumValuation(
                    new Dictionary<string, string>
                    {
                        {"0", "null"},
                        {"1", "one"},
                        {"2", "two"},
                        {"3", "three"}
                    }, _enum1)));
        }
        catch (Exception e)
        {
            Console.WriteLine(e.Message);
        }
        Console.ReadLine();
    }

    private static int GetEnum1(IBaseElement element)
    {
        return _enum1;
    }

    private static void SetEnum1(IBaseElement element, int value)
    {
        _enum1 = value;
    }

    private static int _enum1 = 1;
}

```

8.2.4.6 Add an array data element

```

namespace Sample10;

using System;
using System.Linq;
using ifm.DataStore;
using ifm.Logger;
using ifm.IoTCore;
using ifm.IoTCore.ElementManager.Contracts.Elements;
using ifm.IoTCore.ElementManager.Contracts.Elements.Formats;
using ifm.IoTCore.ElementManager.Contracts.Elements.Valuations;

internal class Program
{
    private static void Main()
    {
        try
        {

```

```

var iotCore = iotCoreFactory.Create("MyIoTCore",
    new DataStore("MyIoTCore.txt"),
    new NullLogger());

var struct1 = iotCore.ElementManager.CreateStructureElement(iotCore.Root,
    "struct1");

var array1 = iotCore.ElementManager.CreateDataElement<int[]>(struct1,
    "array1",
    GetArray1,
    SetArray1,
    format:
        new ArrayFormat(
            new ArrayValuation(
                new Int32Format(new Int32Valuation(0, 100))));
    }
catch (Exception e)
{
    Console.WriteLine(e.Message);
}
Console.ReadLine();
}

private static int[] GetArray1(IBaseElement element)
{
    return _array1;
}

private static void SetArray1(IBaseElement element, int[] value)
{
    if (_array1.SequenceEqual(value)) return;
    _array1 = value;
}

private static int[] _array1 = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];
}

```

8.2.4.7 Add an object data element

```

namespace Sample11;

using System;
using ifm.DataStore;
using ifm.Logger;
using ifm.IoTCore;
using ifm.IoTCore.ElementManager.Contracts.Elements;
using ifm.IoTCore.ElementManager.Contracts.Elements.Formats;
using ifm.IoTCore.ElementManager.Contracts.Elements.Valuations;

internal class Program
{
    private static void Main()
    {
        try
        {
            var iotCore = iotCoreFactory.Create("MyIoTCore",
                new DataStore("MyIoTCore.txt"),
                new NullLogger());

            var struct1 = iotCore.ElementManager.CreateStructureElement(iotCore.Root,
                "struct1");

            var intField = new ObjectValuation.Field("intField1",
                new Int32Format(new Int32Valuation(-100, 100)));
            var floatField = new ObjectValuation.Field("floatField1",
                new FloatFormat(new FloatValuation(-100.0f, 100.0f, 3)));
            var stringField = new ObjectValuation.Field("stringField1",
                new StringFormat(new StringValuation(10, 10, "dd-mm-yyyy")));
            iotCore.ElementManager.CreateDataElement<UserData>(struct1,
                "object1",
                GetObject1,
                SetObject1,
                format: new ObjectFormat(new ObjectValuation([
                    intField,

```

```

        floatField,
        stringField
    )));
    }
    catch (Exception e)
    {
        Console.WriteLine(e.Message);
    }
    Console.ReadLine();
}

private static UserData GetObject1(IBaseElement element)
{
    return _object1;
}

private static void SetObject1(IBaseElement element, UserData value)
{
    if (_object1.Equals(value)) return;
    _object1 = value;
}
private static UserData _object1 = new UserData();
}

internal class UserData : IEquatable<UserData>
{
    public string String1 = "Hallo";
    public int Int1 = 10;
    public float Float1 = 1.2345f;

    public bool Equals(UserData other)
    {
        if (this == other) return true;
        if (other == null) return false;
        return Int1 == other.Int1 &&
            Float1.Equals(other.Float1) &&
            String1 == other.String1;
    }

    public override bool Equals(object obj)
    {
        return Equals(obj as UserData);
    }

    public override int GetHashCode()
    {
        return GetHashCode.Combine(String1, Int1, Float1);
    }
}

```

8.2.5 Add a simple data element

ToDo

8.3 HANDLE ERRORS IN SERVICE AND DATA ELEMENT HANDLER FUNCTIONS

If a service or data element handler function fails to execute it needs to report that error to its caller. The service function can be invoked from the application via the programming API or from a network adapter via the command API. To report the error the function must throw an exception. If the function was invoked via the programming API, the application needs to handle the exception. If the function was invoked via the command API, the response message code is set from the response code in the exception and the additional error information in the response message payload is set from the according exception properties.

The IoTCore application framework provides an exception class called “IoTCoreException” and derived classes which already set the right response code. If an object of this class is thrown the command API takes the information from that object and sets the error information in the response mes-

sage accordingly. If you throw one of the .net system exceptions or if you let a system exception slip through your code, this will result in the return code **500** (Internal error).

8.4 REMOVING AN ELEMENT

To remove an element from the element tree call the “RemoveElement” method from element manager and pass in the parent element and the element you want to remove. All child elements of the removed element will also be removed. The sample also demonstrates how treechanged events are used.

```
namespace Sample12;

using System;
using ifm.Common.Tree;
using ifm.DataStore;
using ifm.Logger;
using ifm.IoTCore;
using ifm.IoTCore.ElementManager.Contracts.Elements;

internal class Program
{
    private static void Main()
    {
        try
        {
            var ioTCore = IoTCoreFactory.Create("MyIoTCore",
                new DataStore("MyIoTCore.txt"),
                new NullLogger());

            // Register treechanged event handler
            ioTCore.Root.TreeChanged += HandleTreeChangedEvent;

            // Create an element and do not raise a treechanged event
            var struct1 = ioTCore.ElementManager.CreateStructureElement(ioTCore.Root,
                "struct1");

            // Raise a treechanged event on demand
            ioTCore.ElementManager.RaiseTreeChanged(TreeChangedActions.ChildAdded,
                ioTCore.Root,
                struct1);

            // Remove an element and raise a treechanged event
            ioTCore.ElementManager.RemoveElement(ioTCore.Root,
                struct1,
                raiseTreeChanged: true);
        }
        catch (Exception e)
        {
            Console.WriteLine(e.Message);
        }
        Console.ReadLine();
    }

    private static void HandleTreeChangedEvent(object sender, EventArgs e)
    {
        var treeChangedEventArgs = (TreeChangedEventArgs<IBaseElement>)e;

        // Handle event
        switch (treeChangedEventArgs.Action)
        {
            case TreeChangedActions.ChildAdded:
                Console.WriteLine("Element added");
                break;
            case TreeChangedActions.ChildRemoved:
                Console.WriteLine("Element removed");
                break;
            case TreeChangedActions.TreeChanged:
                Console.WriteLine("Tree changed");
                break;
        }
    }
}
```

```

        case TreeChangedActions.LinkAdded:
        case TreeChangedActions.LinkRemoved:
            break;
        default:
            throw new ArgumentOutOfRangeException();
    }
}
}
}

```

8.5 GETTING ELEMENTS BY ADDRESS

To get a specific element from the IoTCore element tree call the element manager “GetElementByAddress” method and pass in the element address.

```

namespace Sample13;

using System;
using ifm.DataStore;
using ifm.Logger;
using ifm.IoTCore;

internal class Program
{
    private static void Main()
    {
        try
        {
            var iotCore = IoTCoreFactory.Create("MyIoTCore",
                new DataStore("MyIoTCore.txt"),
                new NullLogger());

            var struct1 = iotCore.ElementManager.CreateStructureElement(iotCore.Root,
                "struct1");

            iotCore.ElementManager.CreateSimpleDataElement<string>(struct1,
                "string1");

            var string1 = iotCore.ElementManager.GetElementByAddress("/struct1/string1");
            Console.WriteLine(string1.Address);
        }
        catch (Exception e)
        {
            Console.WriteLine(e.Message);
        }
        Console.ReadLine();
    }
}

```

8.6 ADD USER SPECIFIC DATA TO ELEMENTS

Each element has a member field of called “UserData” to where application specific data can be hooked up. When the service element calls the service handler, it passes a reference to itself to the service handler as first parameter. Through this reference, the service handler function can access the user data field of the element. This can be useful, e.g. in a service handler function, when additional, element specific information is needed.

```

namespace Sample14;

using System;
using ifm.DataStore;
using ifm.Logger;
using ifm.IoTCore;
using ifm.IoTCore.ElementManager.Contracts.Elements;

internal class Program
{
    private static void Main()
    {

```

```

try
{
    var iotCore = IoTCoreFactory.Create("MyIoTCore",
        new DataStore("MyIoTCore.txt"),
        new NullLogger());

    var struct1 = iotCore.ElementManager.CreateStructureElement(iotCore.Root,
        "struct1");

    var service1 = iotCore.ElementManager.CreateActionServiceElement(struct1,
        "service1",
        HandleService1);
    service1.AddUserData<UserData>("user_data", new UserData());
    service1.Invoke();
}
catch (Exception e)
{
    Console.WriteLine(e.Message);
}

Console.ReadLine();
}

private static void HandleService1(IBaseElement element)
{
    var userData = element.GetUserData<UserData>("user_data");
    Console.WriteLine($"Do something with {userData}");
}
}

internal class UserData
{
    public string String1 = "Hallo";
    public int Int1 = 10;
    public float Float1 = 1.2345f;

    public override string ToString()
    {
        return $"Int1={Int1} Float1={Float1} String1={String1}";
    }
}

```

8.7 SUBSCRIBE TO EVENTS

8.7.1 Add an event handler to the “treechanged” event of the IoTCore

To allow your application to react when the IoTCore element tree changes, you can add an event handler to the “TreeChangedEvent” of the IoTCore root element.

```

namespace Sample15;

using System;
using ifm.Common.Tree;
using ifm.DataStore;
using ifm.Logger;
using ifm.IoTCore;
using ifm.IoTCore.ElementManager.Contracts.Elements;

internal class Program
{
    private static void Main()
    {
        try
        {
            var iotCore = IoTCoreFactory.Create("MyIoTCore",
                new DataStore("MyIoTCore.txt"),
                new NullLogger());

            // Register treechanged event handler
            iotCore.Root.TreeChanged += HandleTreeChangedEvent;

```



```

// Create an element and do not raise a treechanged event
var struct1 = iOTCore.ElementManager.CreateStructureElement(iOTCore.Root,
    "struct1");

// Raise a treechanged event on demand
iOTCore.ElementManager.RaiseTreeChanged(TreeChangedActions.ChildAdded,
    iOTCore.Root,
    struct1);

// Remove an element and raise a treechanged event
iOTCore.ElementManager.RemoveElement(iOTCore.Root,
    struct1,
    raiseTreeChanged: true);
}
catch (Exception e)
{
    Console.WriteLine(e.Message);
}
Console.ReadLine();
}

private static void HandleTreeChangedEvent(object sender, EventArgs e)
{
    var treeChangedEventArgs = (TreeChangedEventArgs<IBaseElement>)e;

    // Handle event
    switch (treeChangedEventArgs.Action)
    {
        case TreeChangedActions.ChildAdded:
            Console.WriteLine("Element added");
            break;
        case TreeChangedActions.ChildRemoved:
            Console.WriteLine("Element removed");
            break;
        case TreeChangedActions.TreeChanged:
            Console.WriteLine("Tree changed");
            break;
        case TreeChangedActions.LinkAdded:
        case TreeChangedActions.LinkRemoved:
            break;
        default:
            throw new ArgumentOutOfRangeException();
    }
}
}
}

```

8.7.2 Add an event handler to an event element

To allow your application to react when an event is raised, you can add an event handler to an event element.

```

namespace Sample16;

using System;
using ifm.DataStore;
using ifm.Logger;
using ifm.IOTCore;
using ifm.IOTCore.Common;
using ifm.IOTCore.ElementManager.Contracts.Elements;

internal class Program
{
    private static void Main()
    {
        try
        {
            var iOTCore = IOTCoreFactory.Create("MyIoTCore",
                new DataStore("MyIoTCore.txt"),
                new NullLogger());

            var data1 = iOTCore.ElementManager.CreateSimpleDataElement(iOTCore.Root,

```

```

        "data1",
        true,
        true,
        true,
        100);
var event1 = (IEventElement)data1.GetElementByIdentifier(Identifiers.DataChanged);
event1.EventRaised += (_, _) =>
{
    Console.WriteLine("event1 raised");
};

data1.Value = 200;
}
catch (Exception e)
{
    Console.WriteLine(e.Message);
}
Console.ReadLine();
}
}

```

8.8 RAISE EVENTS

You can raise an event to inform your application about an incident in your application. To raise an event, call the “RaiseEvent” method of your event element. Data changed events from data elements will be raised automatically when the data in the data element changes.

```

namespace Sample17;

using System;
using System.Threading;
using ifm.DataStore;
using ifm.Logger;
using ifm.IoTCore;
using ifm.IoTCore.ElementManager.Contracts.Elements;

internal class Program
{
    private static Timer _eventTimer;

    private static void Main()
    {
        try
        {
            var iotCore = IoTCoreFactory.Create("MyIoTCore",
                new DataStore("MyIoTCore.txt"),
                new NullLogger());

            var event1 = iotCore.ElementManager.CreateEventElement(iotCore.Root,
                "event1");
            event1.EventRaised += HandleEvent1;

            _eventTimer = new Timer(HandleTimer, event1, 5000, 3000);
        }
        catch (Exception e)
        {
            Console.WriteLine(e.Message);
        }
        Console.ReadLine();

        _eventTimer.Dispose();
    }

    private static void HandleTimer(object state)
    {
        ((IEventElement)state).Raise();
    }

    private static void HandleEvent1(object sender, EventArgs _)
    {

```

```

        Console.WriteLine($"Received event from {(IEventElement)sender}.Address");
    }
}

```

8.9 CREATE LINKS TO ELEMENTS

You can create a link from a source element to a target element. The target element is then accessible via its original address and via its linked address. You can also give the link a meaningful, user friendly name. If you do not provide a name for the link the identifier of the target element will be used. You create a link by calling the “AddLink” method from the element manager and pass in the source element and the target element. To remove the link you call the “RemoveLink” method.

```

namespace Sample18;

using System;
using System.Diagnostics;
using ifm.DataStore;
using ifm.Logger;
using ifm.IoTCore;

internal class Program
{
    private static void Main()
    {
        try
        {
            var iotCore = IoTCoreFactory.Create("MyIoTCore",
                new DataStore("MyIoTCore.txt"),
                new NullLogger());

            var struct1 = iotCore.ElementManager.CreateStructureElement(iotCore.Root,
                "struct1");

            var struct2 = iotCore.ElementManager.CreateStructureElement(struct1,
                "struct1");
            var data1 = iotCore.ElementManager.CreateSimpleDataElement<int>(struct2,
                "data1", value:123);

            // Create a link from root element to data1 element
            iotCore.ElementManager.AddLink(iotCore.Root,
                data1,
                "link_data1");

            // Get the element via link
            var element = iotCore.ElementManager.GetElementByAddress("/link_data1");
            Debug.Assert(element != null);

            // Remove the link from root element to data1 element
            iotCore.ElementManager.RemoveLink(iotCore.Root, data1);
            element = iotCore.ElementManager.GetElementByAddress("/link_data1");
            Debug.Assert(element == null);
        }
        catch (Exception e)
        {
            Console.WriteLine(e.Message);
        }
        Console.ReadLine();
    }
}

```

8.10 LOGGING

To enable logging you first create a logger object and pass it into the IoTCore factory “Create” method. The logger must implement the “ILogger” interface. You can access the logger from the IoTCore object “Logger” property. You can use the logger throughout your application or pass it into other

classes that support the ILogger interface. That way all components of your application share the same log file. The IoTCore application framework provides a logger based on Log4Net.

8.11 PERSISTENCE

The IoTCore provides a persistence manager which implements the “IPersistenceManager” interface. The persistence manager can record and replay service calls. A service that wants to persist when its called can use the persistence manager “Persist” method and pass in the service address and the service data. The method returns an identifier to select the persisted service call. With this identifier a corresponding “undo” service call can remove the previously persisted call. The application can call the “Restore” method to run the previously persisted service call. This is typically done by the application during initialization. Because also services from profiles and services which the application provides can use the persistence manager the application should first load and initialize all profiles and its own services before calling the persistence manager “Restore” method.

ToDo: Add example

8.12 CREATE NETWORK ADAPTERS

8.12.1 Create a server network adapter

To make your IoTCore application accessible from a client over a network you must create a network adapter server and register it with the IoTCore and then start the server. Network adapter servers can support various protocols over various communication interfaces. The standard, what all IoTCores support is JSON over HTTP. When you register the network adapter server with the IoTCore, the IoTCore will take ownership of the object. That means, that the IoTCore is responsible for stopping the server and disposing the object. You can still stop and restart the server as needed, while the IoTCore object is not disposed and implicitly disposed the network adapter server object.

8.12.2 Create a client network adapter factory

To allow the IoTCore to connect as client to other applications over a network you must create a network adapter client factory and register it with the IoTCore. This is necessary, for instance, to enable the IoTCore to send events to another application or to mirror another IoTCore into its own element tree. When the IoTCore needs to connect to another application it uses the network adapter client factory to create a network adapter client for the required communication interface and protocol. With this client it connects to the remote application. When you register the network adapter client factory with the IoTCore, the IoTCore will take ownership of the object. That means, that the IoTCore is responsible for disposing the object.

8.12.3 Create a Http Server Network Adapter

The IoTCore SDK provides an HTTP server network adapter and JSON message converter. To add a server to your application create a new instance and register it with the IoTCore.

```
namespace Sample19;

using System;
using ifm.DataStore;
using ifm.Logger;
using ifm.IoTCore;
using ifm.IoTCore.MessageConverter.Json;
using ifm.IoTCore.NetAdapter.Http.Server;

internal class Program
{
    private static void Main()
    {
```

```

try
{
    const string httpUri = "http://127.0.0.1:8090";
    var logger = new ConsoleLogger();
    var iotCore = IoTCoreFactory.Create("MyIoTCore",
        new DataStore("MyIoTCore.txt"),
        logger);

    logger.Info("Create a JSON message converter for use in network adapters");
    var messageConverter = new MessageConverter(MessageDeserializationMode.ThrowOnNull);

    logger.Info($"Starting HTTP server on {httpUri}");
    var httpServer = new HttpServerNetAdapter(iotCore.MessageDispatcher,
        new Uri(httpUri),
        messageConverter,
        new HttpServerNetAdapterConfiguration(),
        logger, null, null);
    iotCore.ServerNetAdapterManager.RegisterServerNetAdapter(httpServer);
    httpServer.Start();
}
catch (Exception e)
{
    Console.WriteLine(e.Message);
}
Console.ReadLine();
}

```

8.13 CREATE AND LOAD AN IOTCORE PROFILE

The functionality of an IoTCore.Net application can be extended with profiles. A profile is a contract that defines which elements an application has to provide if it wants to support the profile. There are already several ready to use profiles available, some require application specific customization and some do not exist and have to be implemented from scratch by the application developer.

8.13.1 Customize and Load the “deviceinfo” Profile

The “deviceinfo” profile is a mandatory profile, that every IoTCore application must profile. It must be added to the root device element. The “ifm.IoTCore.Profile.DeviceInfo” package contains an implementation of the “deviceinfo” profile, but this has to be customized by the application, because all the information provided by the device info profile is, of course, application specific. To create the profile create a device info provider class that implements the “IDeviceInfo” interface. The “IDeviceInfo” interface defines all properties that the “deviceinfo” profile must provide. Implement the properties according to your application. Then create an instance of the “DeviceInfoProfileBuilder” class and pass in the element manager, the element where the profile is added (this must be the root element) and an instance of your device info provider. Then call the “Build” method, which will add the profile elements to the element tree.

```

namespace Sample20;

using System;
using ifm.DataStore;
using ifm.Logger;
using ifm.IoTCore;
using ifm.IoTCore.Profile.DeviceInfo;

internal class Program
{
    private static void Main()
    {
        try
        {
            var logger = new ConsoleLogger();
            var dataStore = new DataStore("MyIoTCore.txt");
            var iotCore = IoTCoreFactory.Create("MyIoTCore",
                dataStore, logger);

```

```

        logger.Info("Adding profile 'deviceinfo'");
        var deviceInfoProfileBuilder = new DeviceInfoProfileBuilder(ioTCore.ElementManager,
                                                                    ioTCore.ElementManager.Root,
                                                                    new DeviceInfoProvider());

        deviceInfoProfileBuilder.Build();
    }
    catch (Exception e)
    {
        Console.WriteLine(e.Message);
    }
    Console.ReadLine();
}

internal class DeviceInfoProvider : IDeviceInfo
{
    public string DeviceName => "DeviceName";
    public string DeviceClass => "DeviceClass";
    public string ProductInstanceUri => "ProductInstanceUri";
    public string DeviceFamily => "DeviceFamily";
    public string DeviceVariant => "DeviceVariant";
    public byte[] DeviceSymbol => null;
    public byte[] DeviceIcon => null;
    public string DeviceManual => "DeviceManual";
    public string SerialNumber => "serialNumber";
    public string ProductId => "ProductId";
    public string ProductName => "ProductName";
    public string ProductCode => "productCode";
    public string ProductText => "ProductText";
    public string OrderNumber => "OrderNumber";
    public string ProductionDate => "ProductionDate";
    public string ProductionCode => "ProductionCode";
    public string DeviceRevision => "DeviceRevision";
    public string RevisionCounter => "RevisionCounter";
    public string HwRevision => "hwRevision";
    public string HwVersion => "HwVersion";
    public string SwRevision => "SwRevision";
    public string SwVersion => "SwVersion";
    public string BootloaderRevision => "BootloaderRevision";
    public string Vendor => "Vendor";
    public IDeviceInfo.FieldbusTypeEnum? FieldbusType => IDeviceInfo.FieldbusTypeEnum.IoT;
    public string VendorText => "VendorText";
    public string VendorUrl => "VendorUrl";
    public byte[] VendorLogo => null;
    public string ProductWebsite => "ProductWebsite";
    public string SupportContact => "SupportContact";
    public byte[] Icon => null;
    public byte[] Image => null;
    public string Standards => "Standards";
    public string ApplicationSpecificTag { get; set; } = "***";
}

```

8.13.2 Create and Load your own Profile

Implementing a profile actually mean to implement all the elements that the profile requires and add the elements to the element tree. The top level element of the profile must have the profile name in its “profiles” property, so that clients that search the element tree for certain profiles can find it.

The example below implements a profile called “my_profile”. The profile defines just one service which is called “do_it”. The service has request data of type “MyRequestServiceData” and a response data of type “MyResponseServiceData”. The profile is added to the IoTCore root element. The “do_it” service just takes the incoming parameters and returns then in the outgoing parameters.

```

namespace Sample21;

using System;
using System.Collections.Generic;
using ifm.Common.Variant;
using ifm.DataStore;

```

```

using ifm.IoTCore;
using ifm.IoTCore.ElementManager.Contracts;
using ifm.IoTCore.ElementManager.Contracts.Elements;
using ifm.IoTCore.MessageConverter.Json;
using ifm.IoTCore.NetAdapter.Http.Server;
using ifm.Logger;

internal class Program
{
    private static void Main()
    {
        const string httpUri = "http://127.0.0.1:8090";
        var logger = new NullLogger();
        var dataStore = new DataStore("MyIoTCore.txt");
        var ioTCore = IoTCoreFactory.Create("MyIoTCore", dataStore, logger);

        // Create and add my profile
        var myProfile = new MyProfile();
        myProfile.Build(ioTCore.ElementManager, ioTCore.Root);

        // Call profile service via API
        var response = myProfile.Dolt(new MyRequestServiceData("The little red rooster"));
        Console.WriteLine(response);

        // Call service via service element
        var el = (IServiceElement)
            ioTCore.ElementManager.GetElementByAddress("/my_profile/do_it");
        var json = el.Invoke(
            Variant.FromObject(new MyRequestServiceData("The little red rooster")));
        Console.WriteLine(json);

        // Call service via strongly typed service interface
        var el2 = (IServiceElement<MyRequestServiceData, MyResponseServiceData>)
            ioTCore.ElementManager.GetElementByAddress("/my_profile/do_it");
        response = el2.Invoke(
            new MyRequestServiceData("The little red rooster"));
        Console.WriteLine(response);

        // Make service accessible from remote
        var httpServer = new HttpServerNetAdapter(ioTCore.MessageDispatcher,
            new Uri(httpUri),
            new MessageConverter(MessageDeserializationMode.ThrowOnNull),
            new HttpServerNetAdapterConfiguration(),
            logger, null, null);
        ioTCore.ServerNetAdapterManager.RegisterServerNetAdapter(httpServer);
        httpServer.Start();

        Console.ReadLine();
    }
}

internal class MyProfile
{
    public string Name = "my_profile";

    public void Build(IElementManager elementManager, IBaseElement parentElement)
    {
        var profileElement = elementManager.CreateStructureElement(parentElement,
            Name,
            profiles: new List<string> { Name });

        elementManager.CreateServiceElement<MyRequestServiceData, MyResponseServiceData>(
            profileElement,
            "do_it", DoltFunc);
    }

    private MyResponseServiceData DoltFunc(IBaseElement element,
        MyRequestServiceData data)
    {
        {
            return Dolt(data);
        }
    }

    // Make service available on API

```

```

public MyResponseServiceData Dolt(MyRequestServiceData data)
{
    return new MyResponseServiceData(data.Input, data.Input.Length);
}
}

public class MyRequestServiceData
{
    [VariantProperty("in", Required = true)]
    public string Input { get; set; }

    [VariantConstructor]
    public MyRequestServiceData()
    {
    }

    public MyRequestServiceData(string input)
    {
        Input = input;
    }

    public override string ToString()
    {
        return $"Input: {Input}";
    }
}

public class MyResponseServiceData
{
    [VariantPropertyAttribute("out", Required = true)]
    public string Output { get; set; }

    [VariantPropertyAttribute("length", Required = true)]
    public int Length { get; set; }

    [VariantConstructor]
    public MyResponseServiceData()
    {
    }

    public MyResponseServiceData(string output, int length)
    {
        Output = output;
        Length = length;
    }

    public override string ToString()
    {
        return $"{{ out: {Output}, length: {Length} }}";
    }
}

```

As this sample shows the service method can be called from the application either directly or via the service element in the IoTCore tree. By starting an network adapter server the service is also accessible from remote, in this case via http.

The required JSON request would be

```

{
  "code": 10,
  "cid": 10,
  "adr": "/my_profile/do_it",
  "data":
  {
    "in": "The little red rooster"
  }
}

```

The response would be

```

{
  "code": 200,

```



```

"cid": 10,
"adr": "/my_profile/do_it",
"data":
{
  "out": "The little red rooster",
  "length": 22
}
}

```

8.13.3 Create the Monitor Profile

```

namespace Sample22;

using System;
using ifm.DataStore;
using ifm.Logger;
using ifm.IoTCore;
using ifm.IoTCore.MessageConverter.Json;
using ifm.IoTCore.NetAdapter.Http.Client;
using ifm.IoTCore.NetAdapter.Http.Server;
using ifm.IoTCore.NetAdapter.Mqtt;
using ifm.IoTCore.NetAdapter.Mqtt.Profiles;
using ifm.IoTCore.Profile.Monitor;

internal class Program
{
    private static void Main()
    {
        try
        {
            const string httpUri = "http://127.0.0.1:8090";
            var logger = new ConsoleLogger();
            var dataStore = new DataStore("MyIoTCore.txt");
            var ioTCore = IoTCoreFactory.Create("MyIoTCore", dataStore, logger);

            logger.Info("Create a JSON message converter for use in network adapters");
            var messageConverter = new MessageConverter(MessageDeserializationMode.ThrowOnNull);

            logger.Info("Register http client factory");
            ioTCore.ClientNetAdapterManager.RegisterClientNetAdapterFactory(
                new HttpClientNetAdapterFactory(messageConverter, logger));

            logger.Info("Register mqtt client factory");
            var mqttClientFactory = new MqttClientNetAdapterFactory(dataStore,
                messageConverter,
                TimeSpan.FromSeconds(20));
            ioTCore.ClientNetAdapterManager.RegisterClientNetAdapterFactory(mqttClientFactory);

            logger.Info("Create connections structure for connection settings");
            var connectionsElement = ioTCore.ElementManager.CreateStructureElement(ioTCore.Root, "connections");

            logger.Info("Create configuration profile for mqtt client factory");
            var mqttClientFactoryProfile = new MqttClientNetAdapterFactoryProfile(mqttClientFactory, dataStore);
            mqttClientFactoryProfile.Create(ioTCore.ElementManager, connectionsElement);

            logger.Info("Adding profile 'monitor'");
            var monitorProfileBuilder = new MonitorProfileBuilder(ioTCore.ElementManager,
                ioTCore.ClientNetAdapterManager,
                ioTCore.ServerNetAdapterManager,
                ioTCore.EventSender,
                dataStore, logger);
            monitorProfileBuilder.Build(ioTCore.Root);
            monitorProfileBuilder.Resume();

            logger.Info($"Starting HTTP server on {httpUri}");
            var httpServer = new HttpServerNetAdapter(ioTCore.MessageDispatcher,
                new Uri(httpUri),
                messageConverter,
                new HttpServerNetAdapterConfiguration(),
                logger, null, null);
            ioTCore.ServerNetAdapterManager.RegisterServerNetAdapter(httpServer);
            httpServer.Start();
        }
    }
}

```

```
    catch (Exception e)
    {
        Console.WriteLine(e.Message);
    }
    Console.ReadLine();
}
```

8.14 SECURE COMMUNICATION

8.14.1 Https

8.14.2 Mqtts